

Git & GitHub

crée par Josias Nteme By SDA Consulting

Objectif

Ce mini-module donne aux apprenants les commandes Git indispensables pour suivre la formation au quotidien : récupérer les nouveaux contenus du dépôt, versionner leur propre travail et le partager avec le formateur, en toute autonomie et sans risque d'écraser le travail des autres. Apprendre à versionner et partager son travail de manière professionnelle.

Prérequis

- Git installé et configuré (`git --version` fonctionne dans le terminal)
- Identité configurée : `git config --global user.name` et `git config --global user.email`
- Un terminal (invite de commandes, PowerShell, ou terminal macOS/Linux)
- Compte GitHub créé et profil renseigné (photo, bio, ville)
- Git configuré en local avec la même adresse e-mail que le compte GitHub (recommandé)

Ce que vous apprendrez

À l'issue de ce mini-module, chaque apprenant sera capable de cloner ce dépôt de formation, de le maintenir à jour à chaque séance avec `git pull`, et de versionner ses propres réponses aux exercices sur une branche personnelle sans jamais perturber le travail du formateur ou des autres participants. Ces réflexes constituent le socle indispensable de tout travail collaboratif en développement et en science des données.

1. Git et GitHub

Git

Git est un système de contrôle de version installé sur l'ordinateur.

Il permet notamment de :

- suivre les modifications ;
- créer des versions ;
- revenir à une version antérieure ;
- travailler avec des branches.

GitHub

GitHub est une plateforme permettant notamment d'héberger des repositories Git et de collaborer.

2. Les trois zones principales

```
Working Directory
  ↓
Staging Area
  ↓
Repository
```

3. Configuration initiale

```
git config --global user.name "Votre Nom"
git config --global user.email "votre-email@example.com"
```

Vérification :

```
git config --global --list
```

Commande :

```
```bash
git add .
```

place les changements dans la staging area.

Puis :

```
git commit -m "Mon message"
```

enregistre une nouvelle version.

---

---

### # 4. Créer ou récupérer un projet

Créer un repository local :

```
```bash
git init
```

Cloner un repository existant :

```
git clone URL_DU_REPOSITORY
```

Entrer dans le dossier :

```
cd nom-du-repository
```

5. Vérifier l'état

```
git status
```

Cette commande est essentielle.

Elle permet de voir :

- les fichiers modifiés ;
 - les fichiers non suivis ;
 - les changements prêts à être commités.
-

6. Ajouter des fichiers

Un fichier :

```
git add fichier.txt
```

Tous les changements :

```
git add .
```

7. Commit

```
git commit -m "Ajout des exercices de la séance 1"
```

Un bon message explique clairement le changement.

Exemples :

```
Ajout du dataset satisfaction  
Correction exercice statistiques  
Ajout du README semaine 1  
Mise à jour du support Git
```

8. Synchronisation avec GitHub

Récupérer les changements :

```
git pull
```

Envoyer les changements :

```
git push
```

9. Historique

```
git log
```

Version courte :

```
git log --oneline
```

10. Workflow recommandé

Avant de travailler :

```
git pull
```

Pendant le travail :

```
git status
```

Après le travail :

```
git add .  
git commit -m "Description claire"  
git push
```

11. Règle d'or

Ne jamais considérer Git comme une simple étape pour envoyer des fichiers sur GitHub.

Git sert à **documenter l'évolution du travail**.

Chaque commit doit raconter une étape compréhensible du projet.